

VRChat Paper Tablet UI Systeem

Producthandleiding, architectuurdocument en AI-startprompt

Voor StefaniInVR werelden - classroom, cinema, festival, kunstwereld en toekomstige VRChat projecten

Kernidee

Een herbruikbare paper tablet UI voor VRChat. De tablet gebruikt een duidelijke architectuur: manager = waarheid, button = input, visual = lokale weergave. Alleen kleine managerwaardes worden gesynchroniseerd, zodat het systeem geschikt blijft voor Quest en late joiners.

Versie	Datum	Unity	Techniek
1.0 productdocument	2026-05-17	Unity 2022.3.22f1	VRChat SDK + UdonSharp + World-space UI

Dit document is bedoeld voor Nova of elke andere AI-assistent die later aan dit systeem moet werken. Het legt uit wat het systeem doet, hoe het werkt, waarom het zo is gemaakt, waar de synced data staat, en hoe fouten met de Scene Analyzer kunnen worden uitgelezen.

Inhoud

1. Wat dit systeem is
2. Waarom het gemaakt is
3. De hoofdregel: manager = waarheid
4. Huidige hierarchy en prefab-denken
5. Netwerk- en late-join architectuur
6. Managers en scripts
7. Tabs, toggles, sliders, projector en TXL visibility
8. Debuggen met de Nova Scene Tablet Analyzer
9. Veilig uitbreiden naar andere werelden
10. AI-startprompt voor toekomstige chats

1. Wat dit systeem is

De VRChat Paper Tablet UI is een herbruikbaar tablet-/clipboard-systeem voor VRChat worlds. Visueel lijkt het op een papieren clipboard met tabs aan de rechterkant. Technisch is het een world-space UI met UdonSharp managers, knoppen, sliders en lokale visual scripts.

Het systeem is ontstaan voor een classroom / learning world, maar het is niet beperkt tot die ene wereld. Het moet later ook gebruikt kunnen worden in andere StefaniInVR werelden zoals een cinema, festivalwereld, kunstwereld, showcase world of event world.

De huidige tablet bevat tabs voor Video, Settings, Tools, Playlists, Info en Credits. Per wereld mogen tabs, icons, teksten, tools, panels en credits worden aangepast, zolang de onderliggende architectuur hetzelfde blijft.

Belangrijk doel

Gebruikers moeten publieke functies kunnen bedienen via de tablet, zonder dat alle logica admin-only is. Daarom staat de netwerkwaarheid niet op panelen of knoppen, maar op altijd actieve manager-objecten.

2. Waarom dit systeem is gemaakt

Standaard Unity UI in VRChat wordt snel rommelig als elke knop zelf data bewaart. Dat geeft problemen met multiplayer, late joiners, admin-only objecten, uitgeschakelde panels en Quest-performance. Dit systeem is gemaakt om dat op een nette manier te voorkomen.

Ontwerpdoelen

- Een duidelijke, visuele tablet die in VR prettig aanvoelt.
- Een herbruikbare UI-structuur voor meerdere VRChat werelden.
- Late joiners moeten dezelfde status zien als bestaande spelers.
- Alleen kleine waarden syncen: ints, bools en bitmasks.
- Geen hover states, sprites, layouts, renderers of materialwaarden over het netwerk syncen.
- Systeem moet begrijpelijk blijven voor Stef en toekomstige AI-hulp.
- Quest-vriendelijk: zo min mogelijk netwerkspam en zo min mogelijk onnodige Update loops.

3. De hoofdregel: manager = waarheid

De belangrijkste architectuurregel is:

Manager = waarheid. Button = input. Visual = lokale weergave.

Onderdeel	Rol	Mag syncen?
Manager	Bewaart de echte status en stuurt visuals aan.	Ja, als de status globaal moet zijn.
Button / Link	Geeft klik, tabkeuze of slideractie door aan manager.	Nee.
Visual / Applier	Toont lokaal wat de manager zegt: sprites, active states, material floats.	Nee.
Panel / Canvas	Alleen UI-weergave. Mag aan/uit.	Nee.

Gevolg: een knop mag nooit de netwerkwaarheid zijn. Een knop roept alleen een managerfunctie aan. De manager bewaart de staat en zorgt dat late joiners die staat kunnen ontvangen.

4. Huidige hierarchy en prefab-denken

De nieuwste structuur zet alle belangrijke managers onder UIs/Managers. Dat is bewust gedaan zodat synced variabelen niet onder uitgeschakelde panels, admin-only UI of losse visuals staan.

```

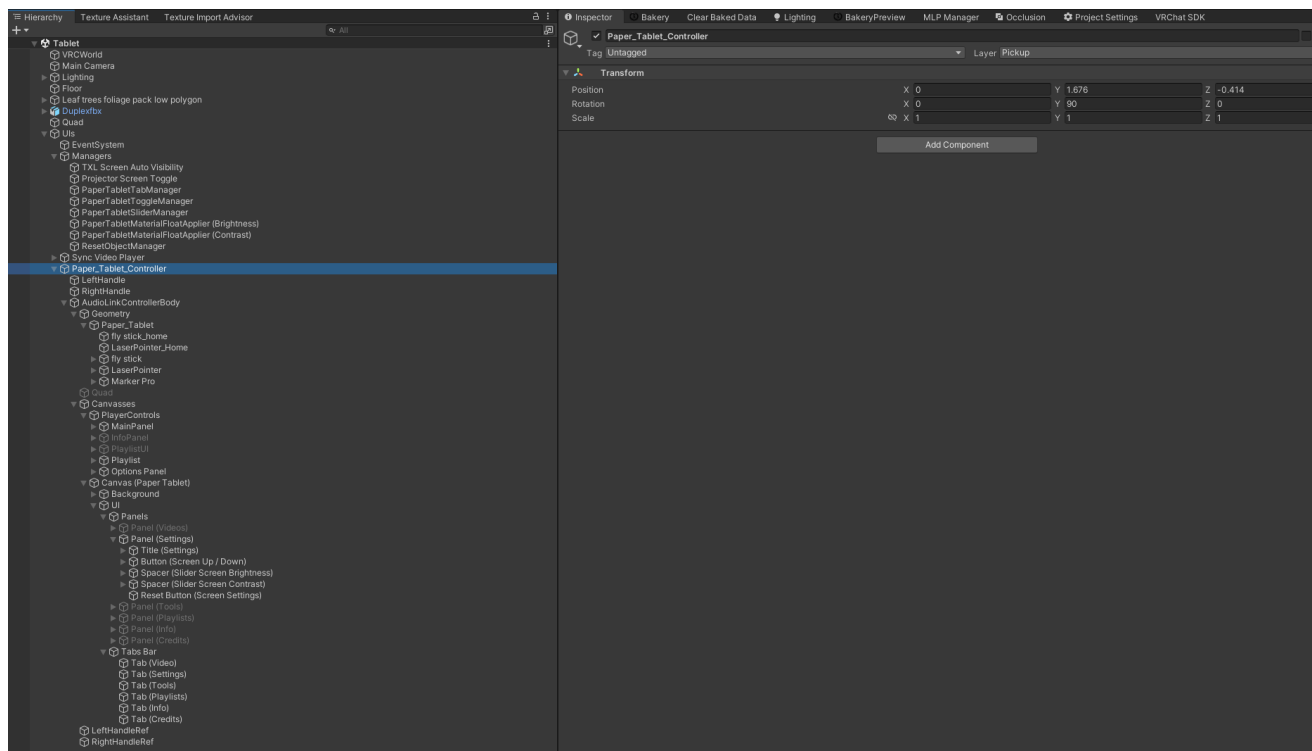
Tablet
| - UIs
|   | - EventSystem
|   | - Managers
|   |   | - TXL Screen Auto Visibility
|   |   | - Projector Screen Toggle
|   |   | - PaperTabletTabManager
|   |   | - PaperTabletToggleManager
|   |   | - PaperTabletSliderManager
|   |   | - PaperTabletMaterialFloatApplier (Brightness)
|   |   | - PaperTabletMaterialFloatApplier (Contrast)
|   |   | - ResetObjectManager
|   | - Sync Video Player
|   | - Paper_Tablet_Controller
|   |   | - LeftHandle
|   |   | - RightHandle
|   |   | - AudioLinkControllerBody
|   |   |   | - Geometry
|   |   |   | - Canvasses
|   |   |   |   | - PlayerControls
|   |   |   |   | - Canvas (Paper Tablet)
|   |   |   |   |   | - Background
|   |   |   |   |   | - UI
|   |   |   |   |     | - Panels
|   |   |   |   |     | - Tabs Bar

```

Altijd actief houden

UIs/Managers en alle manager-objecten daaronder moeten altijd actief blijven. Panels, buttons, visuals en tab-inhoud mogen wel aan en uit worden gezet.

Huidige Unity hierarchy screenshot



Screenshot: de managers staan los boven de visual tablet-controller. Dit is de veilige structuur voor late join en publieke bediening.

5. Netwerk- en late-join architectuur

Het systeem syncet alleen kleine managerwaarden. Visuals worden lokaal opnieuw opgebouwd op basis van die waarden. Hierdoor blijft het licht en Quest-vriendelijk.

Manager	Gesyncte waarde	Type	Waarom klein?
PaperTabletTabManager	selectedTabIndex	int	Alleen het tabnummer wordt gesynct.
PaperTabletToggleManager	stateBits	int bitmask	Maximaal 32 toggles in 1 getal.
PaperTabletSliderManager	value0 t/m value7	int 0..1000	Slider-eindwaarden als kleine integers.
ProjectorScreenToggle	isOpen	bool	Alleen open/dicht status.
ResettableObject	geen compacte manager-state	object sync indien nodig	Fysieke objectreset gebruikt VRObjectSync/ownership.

Niet syncen

- hover state
- pressed state
- sprites
- tekstkleur
- layout
- UI transforms
- sliders als UI-object
- material floats direct
- renderers
- lokale preview tijdens slider slepen

Late join checklist

- Staat de UdonSynced variabele op een manager?
- Staat die manager onder UIs/Managers?
- Blijft UIs/Managers altijd actief?
- Gebruikt de manager Manual sync?
- Roept OnDeserialization een refresh van visuals aan?
- Wordt RequestSerialization pas gedaan nadat ownership klopt?

6. Managers en scripts

Hieronder staat een overzicht van de huidige scripts en hun rol. De complete C# bestanden horen bij het Unity project; deze PDF beschrijft de functie en netwerkarchitectuur.

Script	Sync mode	Gesyncte velden	Belangrijke publieke functies
PaperTabletTabManager	Manual	[UdonSynced] private int selectedTabIndex = 0;	OnDeserialization, SelectTab, GetSelectedTabIndex, IsTabSelected, RefreshAll
PaperTabletTabButton	Unspecified	-	OnButtonPressed

Script	Sync mode	Gesyncte velden	Belangrijke publieke functies
PaperTabletToggleManager	Manual	[UdonSynced] private int stateBits = 0;	OnDeserialization, GetToggleState, IsToggleGlobal, ToggleFromButton, SetToggleState, RefreshAllVisuals
PaperTabletToggleButton	Unspecified	-	OnButtonPressed
PaperTabletToggleVisual	Unspecified	-	RefreshVisual
PaperTabletSliderManager	Manual	[UdonSynced] private int value0 = 500;; [UdonSynced] private int value1 = 500;; [UdonSynced] private int value2 = 1000;; [UdonSynced] private int value3 = 1000;; [UdonSynced] private int value4 = 1000;; [UdonSynced] private int value5 = 1000;; [UdonSynced] private int value6 = 1000;; [UdonSynced] private int value7 = 1000;;	OnDeserialization, GetSliderValueInt, GetSliderValue01, SetSliderValue01, SetSliderValueInt, ResetSlider, ResetAllSliders, RefreshAllAppliers
PaperTabletSliderLink	None	-	OnSliderChanged, CommitSliderValue, RefreshFromManager
PaperTabletMaterialFloatApplier	None	-	RefreshFromManager, ApplyPreview
PaperTabletSliderResetButton	None	-	OnButtonPressed
ProjectorScreenToggle	Manual	private bool isOpen;	ToggleScreen, OnDeserialization, Interact, OnOwnershipTransferred
TXLScreenAutoVisibility	None	-	-
ResetObjectManager	Unspecified	-	ResetEverything, ResetGroup0, ResetGroup1, ResetGroup2, ResetGroup3, ResetGroup4, ResetGroup5, ResetGroup
ResettableObject	Unspecified	-	ResetObject
NovaSceneTabletAnalyzer	Unspecified	-	-

7. Subsystemen

7.1 Tabs

De tabmanager bewaart welke tab actief is. De tabvolgorde is: 0 Video, 1 Settings, 2 Tools, 3 Playlists, 4 Info, 5 Credits. De actieve tab wordt voor iedereen hetzelfde gehouden.

PlayerControls staat aan bij Video en Playlists, en uit bij Settings, Tools, Info en Credits. Extra objecten per tab worden via simpele arrays gekoppeld.

7.2 Toggles

Toggles gebruiken een bitmask in PaperTabletToggleManager. Daardoor kunnen tot 32 globale toggles in 1 integer worden opgeslagen. De knoppen en visuals syncen niet; zij lezen de managerstatus lokaal uit.

Voorbeelden op de Tools tab: Pens, Laser Pointer, Local Sound Control, Coffee, Globe en Fly Tool.

7.3 Sliders

Sliders werken met lokale preview en vertraagde eindwaarde-sync. Tijdens slepen ziet de lokale gebruiker meteen feedback, maar de manager wacht kort en syncet daarna 1 keer. Dit voorkomt continu netwerkverkeer.

Slider	Index	Opslag	Toepassing
Screen Brightness	0	value0, int 0..1000	Material float _Brightness
Screen Contrast	1	value1, int 0..1000	Material float _Contrast

De slider hoeft niet afhankelijk te zijn van Pointer Up. On Value Changed roept PaperTabletSliderLink.OnSliderChanged aan. De manager gebruikt syncDelay om na de laatste beweging te syncen.

7.4 Projector screen toggle

ProjectorScreenToggle staat onder UIs/Managers en bewaart een bool isOpen. De knop Button (Screen Up / Down) roept ProjectorScreenToggle.ToggleScreen aan. De manager stuurt de blendshape van Projector_Screen en de knop-sprites aan.

Dit is belangrijk: de knop bewaart de status niet. De knop is alleen input. De manager is de netwerkwaarheid.

7.5 TXL screen auto visibility

TXLScreenAutoVisibility staat onder UIs/Managers, maar is vooral een lokaal hulpscript. Het kijkt naar de TXL SyncPlayer, het VideoScreen object en de Projector_Screen blendshape. Op basis daarvan zet het zichtbaarheid en colliders goed.

7.6 Reset system

ResetObjectManager staat onder UIs/Managers. ResettableObject staat op objecten die gereset kunnen worden. Voor globale reset van fysieke objecten kan VRObjectSync nodig zijn. Dit is anders dan de compacte manager-state van tabs, toggles en sliders.

8. Debuggen met de Nova Scene Tablet Analyzer

Er is een Editor script gemaakt dat de volledige Unity hierarchy kan uitlezen. Dit is bedoeld om fouten te vinden zonder alles handmatig met screenshots uit te leggen.

Script:

`Assets/Editor/NovaSceneTabletAnalyzer.cs`

Menu in Unity:

`Nova > Scene Tools > Analyze Current Scene`

De analyzer maakt rapporten in:

`Assets/NovaSceneReports/`

Outputformaten:

- TXT
- Markdown
- JSON
- CSV

Wat de analyzer uitleest

- alle GameObjects in de scene
- actieve en inactieve objecten
- hierarchy paths
- components
- serialized fields
- object references
- scripts die in de scene gebruikt worden
- optioneel volledige scriptcode in de output
- missing scripts of lege references

Waarom dit belangrijk is

Als iets niet werkt, kan Stef de analyzer draaien en het .txt of .md bestand aan Nova of een andere AI geven. De AI kan dan precies zien welke manager waar staat, welke references ontbreken, welke objecten actief zijn en welke scripts gekoppeld zijn.

Debugworkflow

- Draai de analyzer in Unity.
- Open Assets/NovaSceneReports.
- Upload of plak het TXT/MD rapport in de chat.
- Laat de AI zoeken naar lege references, verkeerde slider indexen, dubbele scripts, oude button targets en managers die onder verkeerde parents staan.
- Fix alleen de gevonden problemen, niet onnodig de hele architectuur herschrijven.

Scene Analyzer code excerpt

```
[MenuItem("Nova/Scene Tools/Analyze Current Scene")]
```



```
        AnalyzeCurrentScene();
private void AnalyzeCurrentScene()
    string txt = BuildTextReport(scene, allObjects, usedScripts);
    string md = BuildMarkdownReport(scene, allObjects, usedScripts);
    string json = BuildJsonReport(scene, allObjects, usedScripts);
    string csv = BuildCsvReport(allObjects);
private string BuildTextReport(Scene scene, List<GameObject> objects, List<ScriptInfo> usedScripts)
private string BuildMarkdownReport(Scene scene, List<GameObject> objects, List<ScriptInfo> usedScripts)
private string BuildJsonReport(Scene scene, List<GameObject> objects, List<ScriptInfo> usedScripts)
private string BuildCsvReport(List<GameObject> objects)
```

9. Veilig uitbreiden naar andere werelden

De tablet moet als herbruikbaar product worden behandeld, niet als een eenmalige classroom-oplossing.

Wereldtype	Mogelijke tab-/toolinhoud
Classroom	Video, playlists, pens, laser pointer, globe, learning tools.
Cinema / lecture hall	Screen mode, curtain control, talk mode lights, cinema lights.
Festivalwereld	Stage effects, dancefloor lights, info signs, event controls.
Kunstwereld	Guided tour, artwork info, lighting scenes, audio guide.
Showcase world	Prefab demos, credits, options, public controls.

Uitbreidingsregels

- Houd de kernarchitectuur hetzelfde.
- Nieuwe globale status hoort op een altijd actieve manager onder UIs/Managers.
- Gebruik Inspector references; hardcode geen classroom-specifieke objectnamen als dat niet nodig is.
- Maak tabs, panels en tools vervangbaar per wereld.
- Houd credits per wereld aanpasbaar.
- Test altijd local, daarna multiplayer, daarna late join.
- Documenteer nieuwe functies na het testen.

10. AI-startprompt voor toekomstige chats

Kopieer dit blok naar een nieuwe AI-chat wanneer je later verder wilt werken:

Ik werk aan mijn VRChat Paper Tablet UI systeem voor mijn VRChat worlds.
Het systeem is ontstaan voor mijn classroom world, maar moet later ook in mijn andere werelden bruikbaar zijn.

Belangrijk:

- Unity 2022.3.22f1
- VRChat SDK
- UdonSharp
- Quest-vriendelijk
- world-space UI
- managers gebruiken Manual sync
- buttons en visuals gebruiken Sync None
- alleen managers bewaren netwerkwaarheid
- managers staan onder UIs/Managers en moeten altijd actief blijven
- panels, buttons en visuals mogen aan/uit
- late joiners moeten via managerstate de juiste status krijgen

Hoofdregel:

Manager = waarheid.

Button = input.

Visual = lokale weergave.

Huidige managers:

- PaperTabletTabManager syncs selectedTabIndex
- PaperTabletToggleManager syncs stateBits
- PaperTabletSliderManager syncs slider ints 0..1000
- ProjectorScreenToggle syncs isOpen
- ResetObjectManager beheert resets
- TXLScreenAutoVisibility is een lokaal visibility hulpscript

Er is ook een Editor script:

- NovaSceneTabletAnalyzer.cs
- Menu: Nova > Scene Tools > Analyze Current Scene
- Output: TXT, MD, JSON en CSV in Assets/NovaSceneReports
- Doel: hierarchy, references, scripts en fouten uitlezen voor AI-debugging

Werkend:

- tabs
- toggles
- sliders
- projector screen toggle
- TXL screen visibility
- credits/info/playlists/settings/video layout

Help mij vanaf hier met debuggen, polishen of uitbreiden.
Geef bij scripts altijd het volledige script terug.
Verander bestaande werkende scripts zo weinig mogelijk.
Geen gokjes: als iets onzeker is, zeg dat eerlijk.

Appendix A - Script code excerpts

Deze appendix bevat korte code-uittreksels met de belangrijkste sync- en managerstukken. Voor echte wijzigingen moet altijd het actuele .cs bestand uit Unity worden gebruikt.

PaperTabletTabManager

```
[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
public class PaperTabletTabManager : UdonSharpBehaviour
{
    [UdonSynced] private int selectedTabIndex = 0;
    RequestSerialization();
    public override void OnDeserialization()
    public void SelectTab(int tabIndex)
        Networking.SetOwner(Networking.LocalPlayer, gameObject);
    RequestSerialization();
}
```

PaperTabletTabButton

```
public class PaperTabletTabButton : UdonSharpBehaviour
```

PaperTabletToggleManager

```
[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
public class PaperTabletToggleManager : UdonSharpBehaviour
{
    [UdonSynced] private int stateBits = 0;
    RequestSerialization();
    public override void OnDeserialization()
    public void ToggleFromButton(int toggleIndex)
        Networking.SetOwner(Networking.LocalPlayer, gameObject);
    RequestSerialization();
    Networking.SetOwner(Networking.LocalPlayer, gameObject);
    RequestSerialization();
}
```

PaperTabletToggleButton

```
public class PaperTabletToggleButton : UdonSharpBehaviour
```

PaperTabletToggleVisual

```
public class PaperTabletToggleVisual : UdonSharpBehaviour
```

PaperTabletSliderManager

```
[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
public class PaperTabletSliderManager : UdonSharpBehaviour
{
    [UdonSynced] private int value0 = 500;
    [UdonSynced] private int value1 = 500;
    [UdonSynced] private int value2 = 1000;
    [UdonSynced] private int value3 = 1000;
    [UdonSynced] private int value4 = 1000;
    [UdonSynced] private int value5 = 1000;
    [UdonSynced] private int value6 = 1000;
    [UdonSynced] private int value7 = 1000;
    RequestSerialization();
    public override void OnDeserialization()
    public void SetSliderValue01(int sliderIndex, float value01)
    public void SetSliderValueInt(int sliderIndex, int newValue)
        Networking.SetOwner(Networking.LocalPlayer, gameObject);
    RequestSerialization();
    Networking.SetOwner(Networking.LocalPlayer, gameObject);
    RequestSerialization();
}
```

PaperTabletSliderLink

```
[UdonBehaviourSyncMode(BehaviourSyncMode.None)]
public class PaperTabletSliderLink : UdonSharpBehaviour
```

PaperTabletMaterialFloatApplier

```
[UdonBehaviourSyncMode(BehaviourSyncMode.None)]
public class PaperTabletMaterialFloatApplier : UdonSharpBehaviour
```

PaperTabletSliderResetButton

```
[UdonBehaviourSyncMode(BehaviourSyncMode.None)]  
public class PaperTabletSliderResetButton : UdonSharpBehaviour
```

ProjectorScreenToggle

```
[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]  
public class ProjectorScreenToggle : UdonSharpBehaviour  
{  
    [UdonSynced]  
    public void ToggleScreen()  
    {  
        Networking.SetOwner(Networking.LocalPlayer, gameObject);  
        RequestSerialization();  
    }  
    public override void OnDeserialization()  
    {  
    }  
}
```

TXLScreenAutoVisibility

```
[UdonBehaviourSyncMode(BehaviourSyncMode.None)]  
public class TXLScreenAutoVisibility : UdonSharpBehaviour
```

ResetObjectManager

```
public class ResetObjectManager : UdonSharpBehaviour
```

ResettableObject

```
public class ResettableObject : UdonSharpBehaviour  
{  
    Networking.SetOwner(Networking.LocalPlayer, gameObject);  
    Networking.SetOwner(Networking.LocalPlayer, pickupObject.gameObject);  
    Networking.SetOwner(Networking.LocalPlayer, objectSync.gameObject);  
}
```

NovaSceneTabletAnalyzer

```
public class NovaSceneTabletAnalyzer : EditorWindow  
{  
    [MenuItem("Nova/Scene Tools/Analyze Current Scene")]  
    public class SceneReportJson  
    public class GameObjectJson  
    public class ComponentJson  
    public class ScriptInfo  
}
```